The Design and Implementation of Programming Languages

CMSC 323

Prof Szajda

Thanks! This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University)

About me & this class

Prof Szajda

dszajda@Richmond.edu (Rm 219, Jepson Hall)

Lecture

Tuesday, Thursday: 1:30pm – 2:45pm (Weinstein 210)

Lab

Friday: 1:30pm – 2:20pm (Weinstein 210)

Office Hours

TBD (We'll discuss)
And by appointment. Or just drop by.

About you

- Names
- What is your favorite programming language? Why?

Why Programming Languages?

Understanding languages makes you a better programmer

- Different languages share common concepts – operations on basic data, variables and scope, functions, state, etc.
- Understanding these concepts allow you learn new languages more easily
- Differences between languages are variations on these concepts

Why Programming Languages?

Many of you may end up solving language problems … if you know how to recognize them

- The following are languages

  E.g configuration file format – json, xml, yaml etc.

  Data formats – csv, txt

  Template languages - JSX in React

  Custom query language (e.g. “advanced search”)

Why Programming Languages?

Configuration file formats, data formats, templating languages, custom query languages:

- All have variables, datatypes, conditionals, functions etc.
- All need parsers, interpreters, formatters, refactoring and analysis tools and these are all language problems

Why Programming Languages?

If you do not understand the fundamentals of languages, you'll make easily avoidable mistakes

- Strange scoping (Python, JavaScript)
- Inconsistent handling of functions (Ruby)
- Weak type systems (C)
- Lack of expressiveness (most configuration languages), leading to copy and paste (and bugs)

Key Ideas of This Class

Programs are data on which other programs can operate

- To run them (interpreters)
- To transform them (compilers, refactoring tools)
- To check properties about them (type checkers)

Key Ideas of This Class

Programs are data which other programs can use to generate other artifacts

- To make them more efficient (compilers)
- To automate some aspect of programming (code generators)
- To generate infinite test cases (generative testing)

Key Ideas of This Class

Meta-language vs object-language

- Meta-language programs (meta-programs) operate on programs in the object language (object program)
- Our meta-language is a variant of Racket: **#lang plait**
- Our object-language will be many, small and simple – designed to illustrate specific concepts

An Aside: Dr. Racket vs Racket vs Plait

- Dr. Racket: An IDE. Designed specifically for implementing and testing Racket code
- Racket: A programming language derived from the LISP/Scheme family of functional languages.
  - Racket is the language, Dr. Racket it the IDE that comes bundled with it.
- Plait: is a specific "extension" of Racket that is well suited as a meta-language
  - There are differences: E.g., Plait is statically typed, Racket is dynamically typed

Programming Languages

What you'll do in this class:

- Learn how programming languages are designed and built
- Interpreters (written in our meta-language) that executes (object language) programs

Other Concepts Covered

- Local bindings – substitution and deferred substitution
- Multi-arity functions
- Higher-order functions
- Recursions
- State
- Garbage collectors

Programming Languages

What you'll do in this class:

Build other kinds of meta-programs

- Parsers
- Program generators
- Compilers

SMOL

Standard Model of Programming Languages

- eager evaluation
- first-class lexically-scoped functions
- first-order mutable variables, and first-class mutable structures
- safe runtime systems and automated memory management

SMOL

Standard Model of Programming Languages

- eager evaluation
  - Compute the value of an expression as soon as it is bound to a variable or passed to a function
- first-class lexically-scoped functions
  - Functions are treated like any other variable (can be passed to functions, etc). Scoped based on where they appear rather than when they are called

SMOL

Standard Model of Programming Languages

- first-order mutable variables, and first-class mutable structures
  - Variables: basically what you are used to: You create a storage box with a label (the variable) and put a value inside. Later, you can open that box, throw away the old value, and put a new value in its place.
  - Structures: You can treat structures (e.g., lists, dicts, objects) like variables: can pass them to functions, etc.
- safe runtime systems and automated memory management

SImPl

SImPl, the Standard Implementation Plan

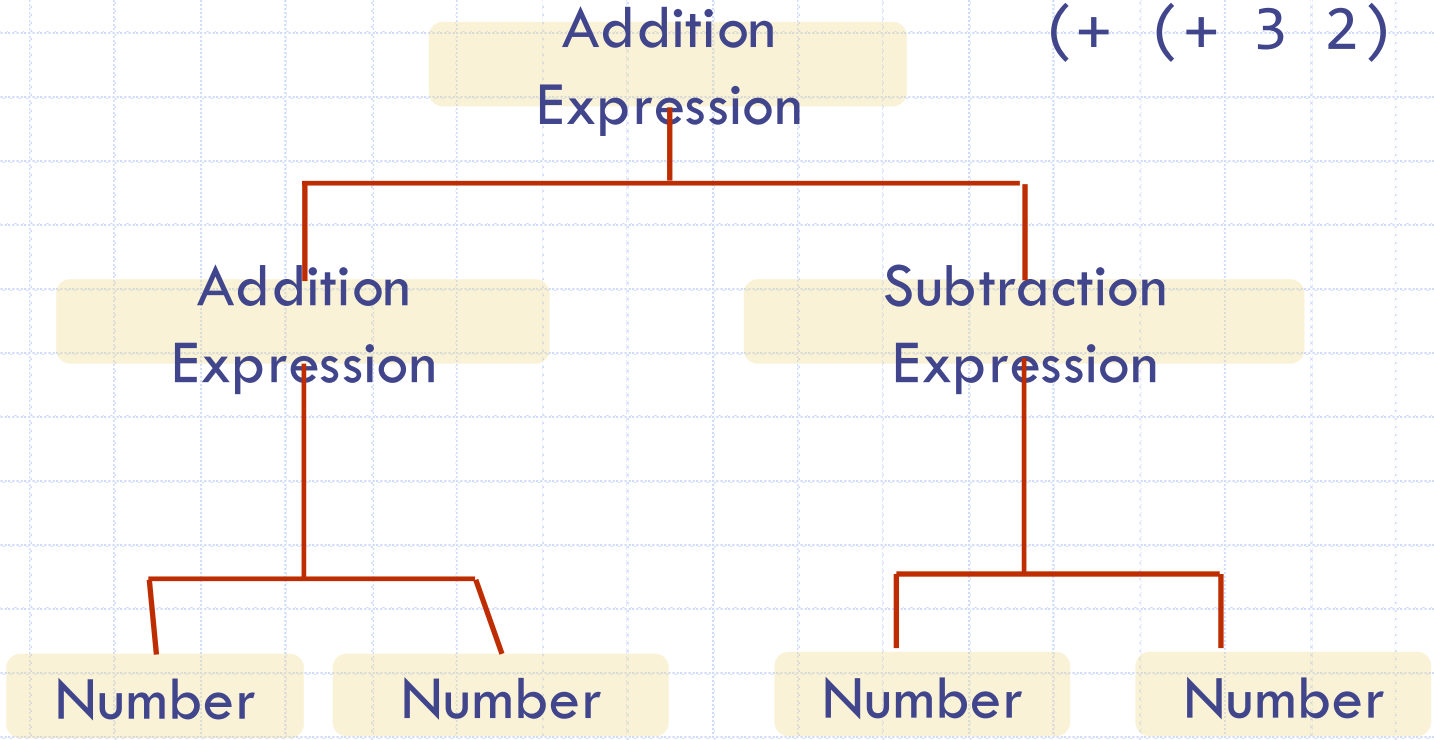
This is the idea that programming languages are usefully thought of in terms of an abstract syntax tree; this tree is represented well by an algebraic datatype; and a program that processes this tree is a recursive function that is largely guided by the structure of the type.

Let's discuss…

Abstract Syntax Tree

Consider the expression

(+ (+ 3 2) (- 3 2))

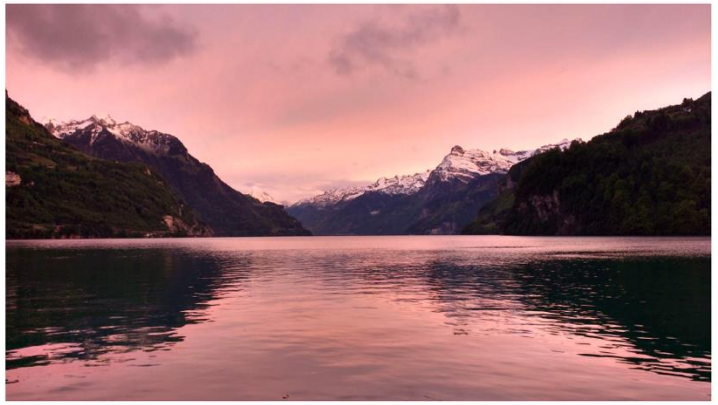


Why PLAIT?

- PLAIT is a language in Racket. Racket is built on scheme which is small in size
- Racket is a language designed for learning about languages
- Racket, scheme, PLAIT are functional languages
- Functional languages are simple and clear (one can map a syntax tree to lines of code easily)

Text

Programming Languages: Application and Interpretation



Shriram Krishnamurthi
Brown University

https://www.plai.org/

Grades

- 7 or 8 programming projects – 70%
- Two midterms (in class) – 20%
- Final exam (in class) – 10%

Programming Projects

- Typically distributed during lab
- Due two weeks later
- Distributed via Github Classroom
  - Assignments are in a personal repository
  - I pull your repository at the due time
    - Late? Costs 10% per day, up to four days

Exercises

- Problems you'll solve in class to prepare you for your programming projects
- Often unannounced and open book
- It's really just practice for what we are learning
  - Not graded

Attendance

- I keep a record of your attendance
- Because we're not an online university, and because we do a lot of practicing and demos during lecture

More Details on all of this

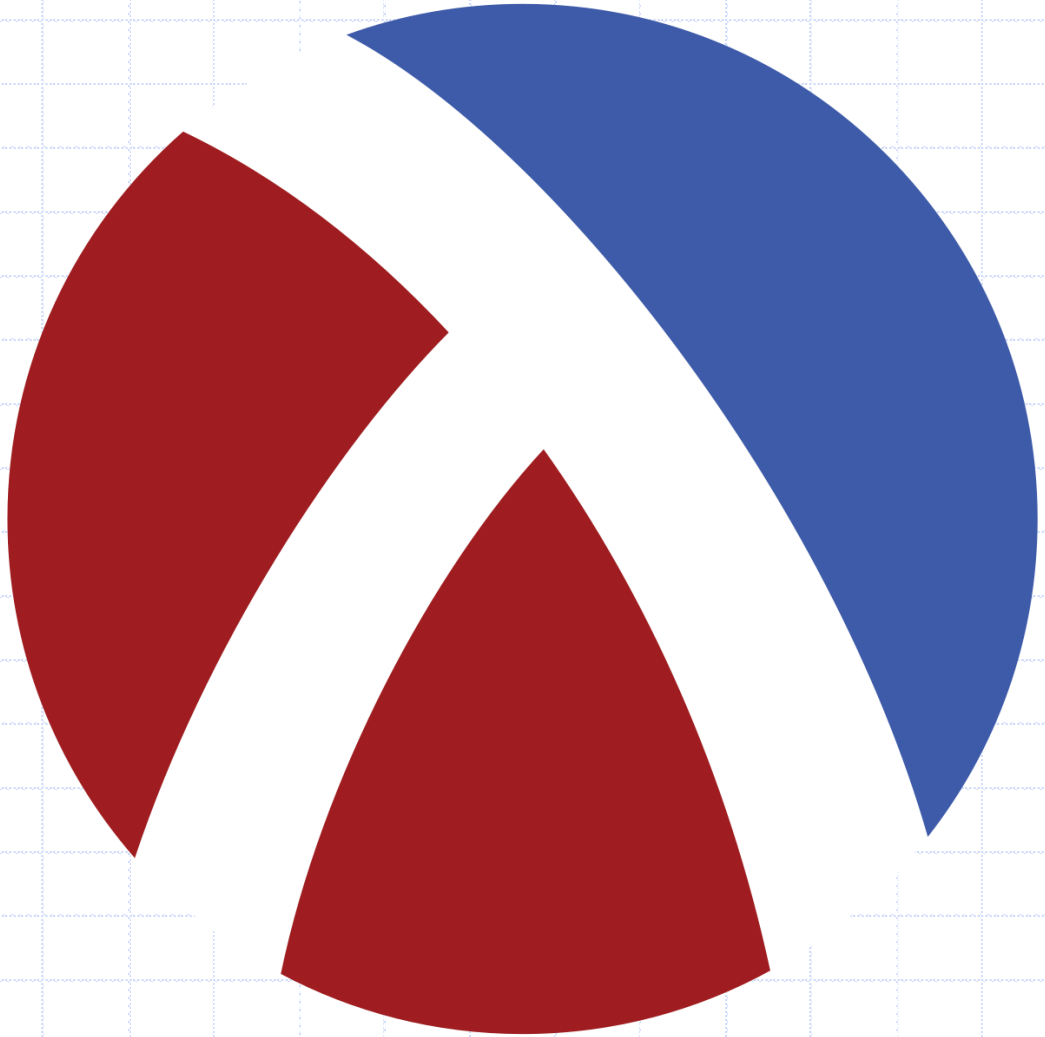
- In the course syllabus

Slack Channel

Channel name: cmsc323-fall-2026

link:https://universityofr-ywo3846.slack.com/archives/C0BSN23RAQZ

Download Racket



https://download.racket-lang.org/

Setup DrRacket

- Open DrRacket
- Go to File

  Install package -> type in “plait”

  wait until close is enabled – click on close

  restart the application

Setup DrRacket

- Open DrRacket
- Set the language to “The Racket Language”

  On the menu bar, select “Language”

  On the submenu, select “Choose Language”

  Choose the option “The Racket Language”

Write and run a test program

```
# lang plait
(+ 2 2)
```

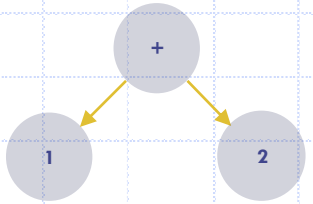
The first line tells Racket that we'll be using the plait language

You can find the documentation for the plait language here:

https://docs.racket-lang.org/plait/

Activity

- Demo factorial
- Demo list functions - first, second, third, fourth, rest and and empty?
- Write a program to add numbers in a list and return the sum



Example

{+ 1 2}

4

Program

AST

Result

Parser

Interpreter

Language used

Example Languages

Object Language

Meta Language

Meta Language

DrRacket

Java

Python

Assembly

Scheme

C, C++ and other languages

C

Machine Code

Scheme

C, C++ and other languages

C

Machine Code

Machine Code